CHAPTER 1

INTRODUCTION

1.1 Preamble

In today's world, digital communication plays a central role in almost every domain, including business, healthcare, education, banking, defense, and social networking. The rapid growth of internet-based communication has increased the volume of confidential data being shared between users. As a result, data security and privacy have become vital concerns. Unauthorized access, data tampering, cyberattacks, identity theft, and surveillance are common threats associated with digital information exchange. Steganography, a branch of information security, deals with concealing secret information within a media file such that its existence remains completely hidden from an observer. Unlike cryptography, where the message becomes scrambled and suspicious to an attacker, steganography ensures that the communication appears normal and harmless. In this project, the focus is specifically on **image steganography**, one of the most widely used and effective forms of steganography due to the large redundancy in image pixels.The aim of this project is to implement a secure, lightweight, and user-friendly system that embeds secret text data into an image using the **Least Significant Bit (LSB) technique**, combined with password-based encryption. This hybrid approach ensures that even if the stego-image is intercepted, the hidden message cannot be retrieved without the correct password. Thus, this project contributes to the field of secure communication by providing a practical and efficient solution for hiding sensitive information inside digital images.

1.2 Motivation

The motivation for undertaking this project arises from several real-world challenges related to data protection:

1. **Increasing Cyber Threats:**
   With cybercrimes rising globally, sensitive data transmitted over the internet is at constant risk of being accessed by unauthorized individuals. Traditional encryption, although secure, is often targeted because encrypted data itself signals importance.
2. **Need for Invisible Communication:**
   Hidden data draws less attention than encrypted data. When a stego-image looks identical to a normal image, adversaries have no reason to suspect its contents.
3. **Simple, Low-Cost Solution:**
   LSB steganography does not require expensive hardware or complex systems. It can run easily on any basic computer or laptop.
4. **Combining Cryptography + Steganography:**
   This provides two layers of security:

   - Encryption protects the *content*,
   - Steganography protects the *existence*.
     This dual protection model is highly beneficial in secure messaging applications.

5. **Practical Applications:**
   Steganography is widely used in fields like:

   - Secure military communication
   - Confidential corporate messaging
   - Protection of patient medical records
   - Digital watermarking
   - Copyright protection
     The simplicity of the LSB method makes it suitable for student projects, research, and industry-level applications.

These factors motivated the design and development of a system that is secure, reliable, lightweight, and easy to implement.

1.3 Problem Statement

With the increase in cyberattacks, traditional encryption alone is not sufficient because encrypted text is easily recognizable and may attract hackers. There is a need for a technique that not only secures the data but also hides the very existence of the communication.

Problem Statement:
To develop a secure data transfer over internet using image steganography system capable of embedding password-encrypted text inside a digital image using Least Significant Bit (LSB) substitution, ensuring high imperceptibility, accurate extraction, and resistance against unauthorized access.

This project aims to address:

- How to hide a secret message without altering image quality noticeably.
- How to ensure the message cannot be retrieved without authentication.
- How to create a lightweight system suitable for real-time secure communication.

1.4 Objectives of the project

1. To design a secure text-hiding mechanism using image-based LSB steganography

The first objective of this project is to develop a method for concealing confidential text within digital images using the Least Significant Bit (LSB) steganography technique. This includes selecting an appropriate color channel, determining optimal bit-alteration strategies, and ensuring that the hidden data is embedded without causing perceptible changes to the visual content of the image.

2. To apply encryption (password-based or FERNET cipher) to ensure message confidentiality

Before embedding, the text message will be encrypted using a password-based approach or a strong cryptographic technique such as the FERNET cipher. This ensures that even if an attacker extracts the hidden data, it remains unreadable without the correct key or password. The integration of encryption strengthens the security model by combining cryptography with steganography.

3. To embed encrypted data into an image in a way that does not visibly distort the original image

A key objective is to maintain the visual integrity of the cover image. The embedding algorithm must ensure that the modifications made to the pixel values are minimal and completely imperceptible to the human eye. This includes selecting appropriate embedding positions, capacity optimization, and ensuring that pixel values remain within valid ranges.

4. To extract and decrypt hidden messages accurately without data loss

The project aims to build a reliable decoding mechanism capable of retrieving the embedded ciphertext from the stego image without corruption. Once extracted, the encrypted data must be successfully decrypted using the correct password or key, ensuring 100% accuracy of the recovered message.

5. To implement a robust, efficient, and user-friendly system that works with minimal hardware requirements

Another objective is to develop the system in such a way that it runs smoothly on standard computing devices without requiring high processing power or specialized hardware. The user interface or workflow should be simple, intuitive, and easily operable even by non-technical users.

6. To evaluate the performance of the system using various quality and efficiency metrics

The performance of the developed steganography system will be analyzed using quantitative and qualitative evaluation measures, including:

- PSNR (Peak Signal-to-Noise Ratio): To measure the visual quality of the stego image compared to the original.
- MSE (Mean Squared Error): To compute the level of pixel distortion introduced during embedding.
- Encoding and decoding time: To determine the computational efficiency of the system.
- Imperceptibility and accuracy: To assess how well the hidden data remains invisible and how accurately it can be retrieved.er

7. To create a practical solution for secure communication and digital privacy

The ultimate goal of the project is to deliver a real-world applicable system that enhances secure communication. The solution can be used in various scenarios such as sharing confidential information, protecting sensitive documents, safeguarding personal messages, and ensuring privacy in digital communication platforms.

1.5 Organization of the report

The report is structured logically to guide the reader through the project development process:

- Chapter 1: Introduction
  Provides the background, motivation, problem definition, objectives, and report structure.
- Chapter 2: Literature Survey
  Reviews classical, deep learning, and coverless steganography research works.
  Includes analysis of 8 recent research papers (2024–2025) along with outcomes.

- Chapter 3: Methodology and System Design
  Explains system architecture, flow diagrams, block diagrams, hardware/software requirements, data flow, and algorithmic steps.
- Chapter 4: Implementation, Results, and Performance Analysis
  Shows actual outputs (stego-images, extracted messages), along with PSNR values, encryption results, speed measurements, and visual comparisons.
- Chapter 5: Conclusion and Future Scope
  Summarizes achievements and proposes extensions like GAN-based steganography, diffusion models, reversible steganography, and mobile/web integration.
- References
  Lists all cited research papers, tools, websites, and technical documentation.

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction

A literature survey forms the backbone of any research-oriented project, as it provides a structured understanding of the developments, limitations, and research gaps that shape the proposed system. In the field of information hiding, steganography—particularly image steganography—has remained an active and rapidly evolving domain of study for more than two decades. Beginning with early classical spatial-domain techniques such as simple Least Significant Bit (LSB) substitution in the late 1990s, the field has progressively matured to include more robust JPEG-domain and transform-domain algorithms that focus on improved imperceptibility and resistance to detection. Over the years, researchers have introduced sophisticated models that leverage mathematical transforms, cost optimization, and adaptive embedding strategies to strengthen security. With the rise of artificial intelligence, modern approaches have significantly shifted toward deep learning–based architectures capable of learning complex embedding patterns. This includes convolutional neural networks (CNNs) for feature-driven hiding, generative adversarial networks (GANs) for realistic stego-image synthesis, and most recently, diffusion models that enable training-free or coverless steganography. Additionally, multimodal and content-aware methods now enable hiding data across images, text, and other media without visibly altering the cover content. Alongside these advancements, the progression of steganalysis techniques—ranging from statistical detectors to deep-learning-based classifiers—has pushed researchers to continuously improve embedding strategies to maintain undetectability.It also includes an in-depth discussion of eight recent research papers (2024–2025), providing insight into current trends, challenges, and technological advancements that shape the future direction of secure image steganography.

2.2 Literature Review

The paper "A Survey on Deep-Learning-Based Image Steganography" by Song et al. (2024), published in *Expert Systems with Applications* and authorized by Elsevier through ScienceDirect, presents an extensive review of how modern deep-learning techniques are being applied to image steganography. It systematically categorizes recent advancements into major architectural groups such as encoder–decoder networks, generative adversarial networks (GANs), autoencoders, and transformer-based models, explaining how each class contributes to improved data-hiding capacity, higher imperceptibility, and stronger robustness against various detection attacks. The authors also emphasize several ongoing challenges in the field, including the need for large and diverse training datasets, the high computational cost of deep models, and the limited generalization of these systems when used with unseen image types or in the presence of distortions and real-world noise. Moreover, the survey highlights the rising significance of reversible and cross-domain steganography, methods that allow hidden data to be fully recovered without compromising image quality. In conclusion, the paper identifies promising future research directions such as lightweight deep-learning architectures suitable for low-resource environments, adversarially robust steganographic models, and approaches focused on practical, real-world deployment. Overall, the survey offers valuable insights into the evolution, limitations, and emerging opportunities in deep-learning-driven image steganography.

The paper "DiffStega: Image Steganography with Diffusion Models" (2024) by Yang et al., presented at IJCAI-24, introduces DiffStega, a novel *training-free* and *coverless* image steganography framework that leverages the generative power of diffusion models. Unlike traditional steganographic techniques that embed secret information inside a pre-existing cover image, DiffStega entirely avoids the use of a cover by generating stego images directly through a controlled diffusion process. The method incorporates a password-dependent reference image as an additional image prompt—alongside public text prompts—ensuring that only users with the correct password can

reproduce or recover the hidden data. To further enhance security, the authors propose a new Noise Flip mechanism, which disrupts unauthorized attempts at reverse engineering or extraction. DiffStega is evaluated on a custom dataset specifically designed for coverless steganography, demonstrating superior adaptability, stronger password sensitivity, and improved data recovery quality compared to previous methods. Its training-free design significantly reduces computational and resource overhead, making it more efficient than many deep-learning-based approaches. Overall, DiffStega represents a major advancement by integrating coverless steganography, diffusion-driven image generation, and password-based protection into a secure and highly stealthy communication framework.

The paper "Robust Coverless Image Steganography Based on Human Pose Estimation" by Tan et al. introduces an innovative coverless steganography approach that uses human pose estimation to transmit hidden information without altering any cover image pixels. The method treats images containing human subjects as natural carriers and employs a pose estimation network to extract stable, high-level semantic features such as skeletal keypoints and joint coordinates. Secret data is then encoded by mapping the information onto the pose features of selected carrier images, ensuring that the images themselves remain visually unchanged. Because the technique avoids pixel-level embedding, it offers strong inherent resistance to conventional steganalysis tools and image-alteration attacks, achieving reported robustness rates of up to ~92% across various public datasets. The system also benefits from the abundance of human-centric images on social media platforms, making the selection of suitable carriers more convenient and realistic. Despite its strengths, the method faces practical limitations, particularly its dependency on images with clearly detectable human poses; performance decreases when carriers include heavy occlusions, blurred subjects, or lack distinct pose cues. Overall, the study demonstrates a promising direction for secure and stealthy coverless steganography using human-pose semantics.

The paper "Design of an Efficient Multimodal Image Steganography" by Ekta and Singh presents a novel multimodal steganography framework that combines multi-domain feature analysis with visual cryptography using Shamir's (n, k) secret-sharing scheme. Unlike conventional methods limited to single media types, the proposed system supports embedding text, images, audio, and video, utilizing resizing and bit-wise operations along with secret-sharing to securely distribute the payload among multiple shares. Experimental evaluations demonstrate notable improvements in performance metrics, including 45%–48% reductions in RMSE, lower MAE, and significantly higher PSNR values compared with earlier approaches, indicating superior imperceptibility and reconstruction quality. By leveraging both spatial and transform-domain features, the framework enhances robustness against distortions and steganalysis, while the inclusion of secret-sharing strengthens security by ensuring that the hidden data can be recovered only when the required shares are combined. Despite its strengths, the approach introduces additional embedding complexity and computational overhead, particularly when handling diverse media modalities. Overall, the study contributes a flexible and secure blueprint for multimodal image steganography, advancing the state of the art in multimedia data hiding.

The paper "JPEG Image Steganography With Automatic Embedding Cost Learning" by Yang et al. introduces JS-GAN, a novel JPEG-domain steganography framework that automatically learns an embedding cost function through a GAN-based architecture, addressing the increasing effectiveness of modern CNN-driven steganalysis. Unlike traditional heuristic cost formulations, JS-GAN adaptively determines embedding costs in a content-aware manner and further enhances robustness through an Estimated Side Information (ESI) module, which introduces asymmetric cost adjustment to better resist detection. Experimental evaluations show that JS-GAN improves the detection error rate by approximately 2.58% compared to the widely used J-UNIWARD method, while its enhanced version, JS-GAN(ESI), achieves an 11.25% improvement under GFR-based steganalysis at QF = 75 and 0.4 bpnzAC. These results highlight the model's superior

resistance to both classical and deep-learning steganalyzers. However, the approach relies on computationally intensive GAN training and domain-specific side-information estimation, which may limit its applicability in lightweight or real-time systems. Overall, the work advances JPEG-domain steganography by integrating automatic cost learning and side-information refinement for improved security and adaptability.

The paper "Digital Image Steganography and Reversible Data Hiding" by Ragab et al. presents a comprehensive survey that bridges the domains of traditional image steganography and reversible data hiding (RDH). The authors explain that while steganography focuses on concealing secret information within images without perceptible distortion, RDH extends this capability by enabling complete recovery of the original cover image after the hidden data is extracted. The survey reviews a wide spectrum of algorithms—including spatial-domain, transform-domain, encryption-based, and compressed-domain techniques—and evaluates them using key performance metrics such as embedding rate, visual quality, payload capacity, and security. It further examines major application areas such as medical imaging, thermal imaging, and military intelligence, where lossless cover recovery is essential for reliability and trust. The authors also outline significant challenges in the field, including achieving high embedding capacity without compromising visual quality, designing efficient reversible schemes for encrypted or compressed images, and improving resistance to modern steganalysis attacks. The paper concludes with actionable future research directions, emphasizing the need for lightweight RDH methods, cross-domain embedding strategies, and better optimization of trade-offs among capacity, imperceptibility, and recoverability.

The paper "Comparative Deep Learning Models in Applications of Steganography Detection" examines various deep-learning architectures for identifying stego images produced by widely used embedding algorithms such as WOW, S-UNIWARD, and HUGO. The authors develop and train a unified Deep Convolutional Neural Network

(Deep-CNN) on datasets containing both cover and stego images, enabling a comparative analysis of detection accuracy across different steganographic schemes. Their findings reveal that although deeper and fine-tuned models outperform traditional shallow networks, performance significantly declines when the test images are generated using embedding algorithms different from those used during training—a challenge known as the algorithm-mismatch problem. This outcome underscores the need for universal steganalysis models capable of detecting unknown or unseen embedding techniques rather than relying solely on training-specific patterns. The study also highlights practical constraints, including the requirement for large, well-balanced datasets and the difficulty of generalizing across diverse embedding strategies and cover image sources. Overall, the work contributes valuable empirical insights to steganalysis research and reinforces the importance of developing more adaptable and comprehensive deep-learning-based detectors.

The paper "Steganography Algorithm Based on Image Retrieval with SURF Features" by Li et al. introduces a novel coverless steganography framework that relies on keypoint-feature retrieval using the Speeded-Up Robust Features (SURF) algorithm. Rather than embedding secret information by modifying pixel values in a cover image, the proposed method retrieves images from synchronized datasets available at both the sender and receiver ends, mapping segments of the secret message to corresponding SURF feature–hash sequences. By selecting images whose feature-hash values match specific secret bit patterns, the scheme avoids any form of pixel-level alteration, greatly enhancing stealth and resisting traditional steganalysis techniques. Moreover, the approach eliminates the need to transmit auxiliary embedding parameters—such as block indices or embedding keys—thereby minimizing the risk of exposure or interception. Experimental results demonstrate superior capacity, robustness, and security compared to earlier coverless steganographic methods. However, the framework's dependence on synchronized datasets for both communicating parties, along with the challenges of maintaining reliable SURF feature matching under diverse

imaging conditions, presents practical limitations. Overall, the work advances the field of coverless steganography by leveraging feature-based image retrieval to achieve high concealment and secure message mapping.

2.3 Outcome of Literature survey

The literature on image steganography indicates a clear progression from traditional spatial-domain techniques to modern deep learning methods. Spatial-domain approaches, such as LSB substitution, offer high embedding capacity but suffer from low robustness against attacks and noise. In contrast, transform-domain methods like DCT and DWT provide greater resistance to compression and image processing while maintaining reasonable imperceptibility, though often at the cost of lower payload. Recent advances leverage deep learning, using CNNs and GANs to achieve improved imperceptibility, adaptability, and content-aware embedding; however, these approaches demand significant computational resources. Common evaluation metrics include PSNR, SSIM, and BER, which assess imperceptibility, payload capacity, and robustness. Despite these developments, balancing capacity, security, and robustness remains a persistent challenge. Current trends focus on hybrid and adaptive models, adversarial training, and content-aware strategies, underscoring the ongoing need for steganographic methods that are secure, efficient, and generalizable across diverse image types.

CHAPTER 3

METHODOLOGY & IMPLEMENTATION

3.1 Introduction

This chapter presents the overall methodology that guided the development and implementation of the project "Secure Data Transfer Over the Internet Using Image Steganography." The methodology outlines the systematic and structured process followed from the initial conceptualization phase to the final execution and evaluation of the system. It provides a comprehensive explanation of the proposed system model, architectural design, algorithms and techniques employed, hardware and software requirements, workflow processes, and detailed implementation procedures. In modern communication environments, increasing concerns regarding security, privacy, and protection of sensitive data have highlighted the limitations of relying solely on traditional encryption techniques, especially in scenarios where encrypted information can be easily detected, flagged, or intercepted by adversaries. Steganography addresses these challenges by embedding secret information into ordinary-looking media—such as digital images—thereby concealing the very existence of the data. When combined with cryptographic algorithms, the resulting hybrid approach enhances security significantly, ensuring that even if the presence of hidden information is suspected, extracting or interpreting it becomes computationally difficult without the appropriate encryption keys. The methodology developed for this project therefore focuses on ensuring confidentiality, integrity, robustness, and invisibility of the embedded data, offering a secure and reliable medium for information transmission. The subsequent sections delve deeper into the conceptual framework, development stages, system components, and the step-by-step procedures followed throughout the design, implementation, and testing of the proposed secure data transfer system.

3.2 Proposed Model

The proposed model aims to implement a secure communication system that embeds encrypted data inside a digital image using the Least Significant Bit (LSB) method. The model is designed to ensure that:

- The message remains confidential through AES encryption.
- The message remains undetectable through steganography.
- The communication remains secure even if intercepted.

The system operates in two major phases:

- Encoding Phase (Sender Side)
- Decoding Phase (Receiver Side)

Each phase integrates cryptography and steganography to provide a robust communication mechanism.

3.2.1 System Objectives of the Proposed Model

The primary objectives of the proposed system design are:

- To create a method of communication that hides crucial data in images.
- To avoid visual distortion in the output image.
- To ensure the hidden data cannot be extracted without authentication.
- To provide a low-cost, platform-independent, software-based solution.
- To facilitate secure communication over any digital medium (email, messaging apps, social networks).

3.2.2 System Architecture

The architecture consists of input components, processing units, and output components.

A. Sender Side Architecture

1. Input Message Acquisition

   The user enters the secret text that needs to be transmitted.

2. Password Input for Encryption
   The sender provides a password which is later converted into an AES key using a hashing algorithm (SHA-256).
3. Encryption Phase
   The message is encrypted, ensuring it cannot be understood by unauthorized individuals.
4. Binary Conversion Phase
   Encrypted text is converted into binary digits for embedding into the image.
5. Image Processing Phase
   The cover image is loaded and processed pixel-wise.
6. LSB Embedding Phase
   The binary data is embedded into the least significant bits of RGB pixel values.
7. Stego-image Generation
   The output image appears visually identical to the original but contains secret data hidden inside it.
8. Transmission Phase
   The sender transmits the encoded image through any communication medium.

B. Receiver Side Architecture

1. Reception of Stego-image
   The encoded image is received through the selected platform.
2. LSB Extraction Phase
   The least significant bits of each pixel are extracted to reconstruct the embedded binary data.
3. Binary Reconstruction Phase
   The extracted bits are grouped to form encrypted text.
4. Password Authentication
   The receiver enters the password provided by the sender.

5. Decryption Phase

   AES decryption algorithm converts ciphertext back to readable plaintext.

6. Output Message Presentation

   The original message is displayed to the receiver.

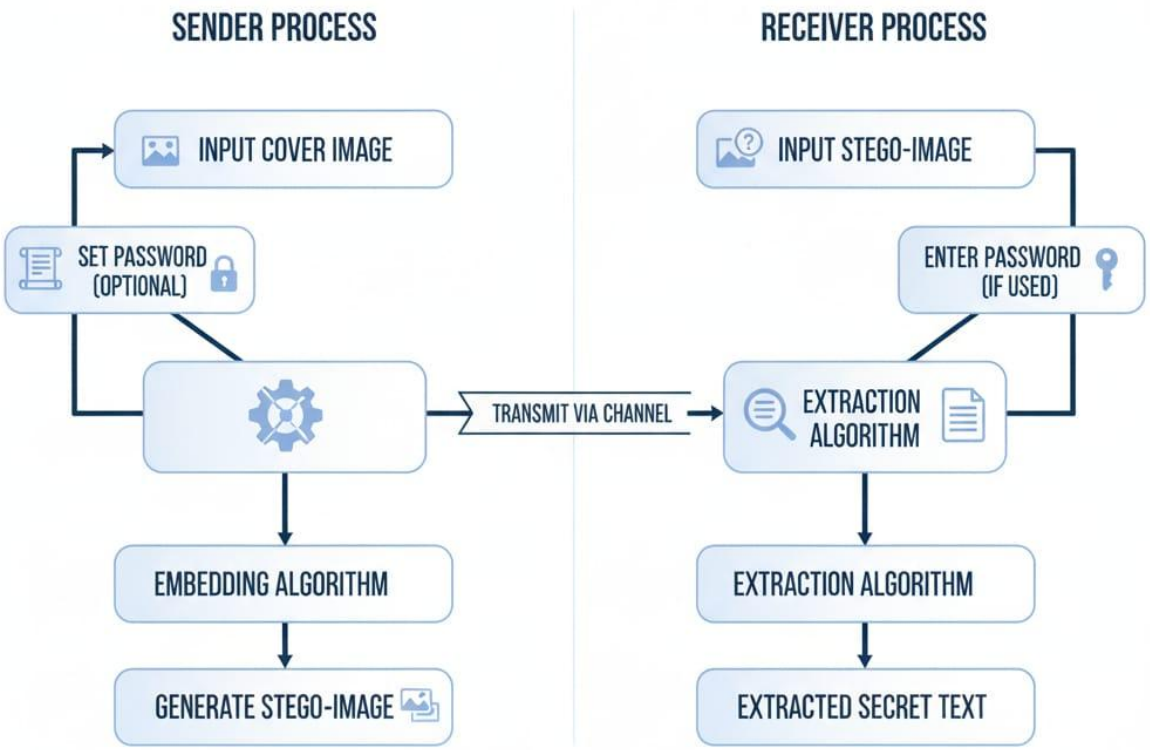

Figure 3.2.2: Block Diagram of Flow

3.3.1 Hardware Requirements

- Laptop/Computer
  - Minimum 4GB RAM
  - Dual-Core CPU
  - Required to run Python environment and perform image-processing operations.
- Storage
  - At least 500MB of free disk space
  - Needed for storing source code, images, and output files.
- Internet Connection
  - Basic broadband connection
  - Used for transmitting encoded images.
- Peripheral Devices
  - Keyboard and Mouse
  - For general user interaction.
- Note:
  - No specialized hardware is required.
  - The project is lightweight and fully software-based.

3.3.2 Software Requirements

- Python 3.x
  - Primary programming language
  - Used for implementing all core functionalities.
- Pillow (PIL)
  - Python imaging library
  - Enables reading, editing, and writing image pixel data.
- NumPy
  - Numerical computing library

- Supports efficient array and pixel-level manipulation.

- Cryptography (AES)
  - Python cryptography framework
  - Provides secure AES-based message encryption.
- VS Code / PyCharm
  - Code editors / IDEs
  - Used for development, testing, and debugging.
- Operating System
  - Compatible with Windows, macOS, and Linux
  - Provides the execution environment.
- Note:
  - The entire software stack is open-source, free, and cost-effective.

3.4 Design and Implementation

The design methodology follows a structured and systematic workflow that ensures the development of a secure and efficient steganography system. It begins with requirement analysis, where the project's goals, security expectations, and operational constraints are clearly defined. This is followed by system modeling, which involves planning and structuring the sender and receiver modules to establish smooth data flow and functional clarity. Next, algorithm formulation focuses on defining the logic for embedding and extracting hidden messages using LSB-based steganographic techniques. After the conceptual groundwork, the coding phase implements these algorithms using Python and supporting libraries to create a functional system. Once implemented, testing and validation are carried out to assess accuracy, ensure that the hidden data remains invisible within the image, and verify security features such as encryption integrity. Finally, the process moves into optimization, where performance is refined, embedding efficiency is improved, and image distortion is minimized to produce a reliable and high-quality steganographic system.

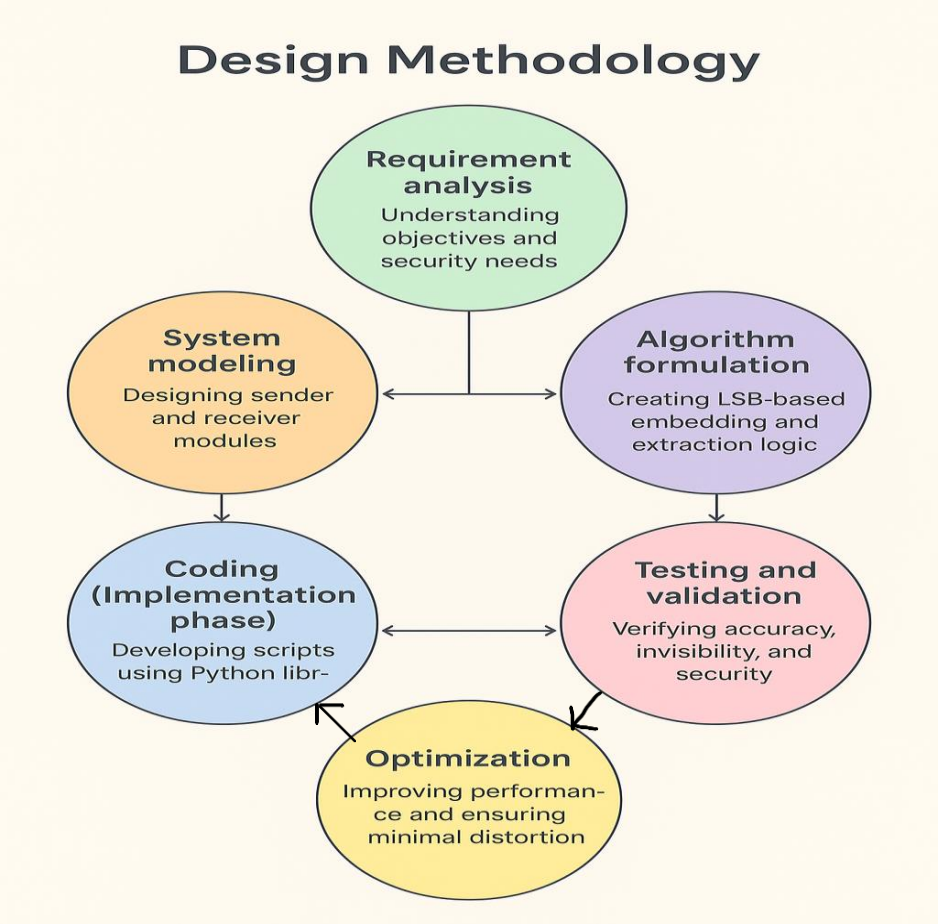

Figure 3.4.1: Proposed Methodology

3.4.2 Implementation

A. IMPLEMENTATION OF ENCODING (SENDER SIDE)

File: secure_encode.py

- **Step 1: Import Necessary Libraries**

Code:

```
from PIL import Image
import numpy as np
from Crypto.Cipher import AES
```

```
import hashlib
import base64
```

Explanation:

- PIL (Pillow): Used to open, read, modify, and save images.
- NumPy: Converts the image into arrays for pixel-level manipulation.
- AES (PyCryptodome): Provides encryption and decryption functions.
- Hashlib: Converts the password into a secure AES key using SHA-256.
- Base64: Encodes encrypted data so it becomes text-friendly.

Figure 3.4(1): Input Image

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\CHARANYA B\OneDrive\Desktop\SecureImageSteganography> & C:\Python314\python.exe "c:/Users/CHARANYA B/OneDrive/Desktop/SecureImageSt
eganography/src/secure_encode.py"
Enter input image file name:
```

Figure 3.4(2): Encoding Terminal

```
PS C:\Users\CHARANYA B\OneDrive\Desktop\SecureImageSteganography> & C:\Python314\python.exe "c:/Users/CHARANYA B/OneDrive/Desktop/SecureImageSt
eganography/src/secure_encode.py"
Enter input image file name: C:\Users\CHARANYA B\OneDrive\Desktop\SecureImageSteganography\images\input_image.png
```

Figure 3.4(3): input image file path

```
PS C:\Users\CHARANYA B\OneDrive\Desktop\SecureImageSteganography> & C:\Python314\python.exe "c:/Users/CHARANYA B/OneDrive/Desktop/SecureImageSt
eganography/src/secure_encode.py"
Enter input image file name: C:\Users\CHARANYA B\OneDrive\Desktop\SecureImageSteganography\images\input_image.png
Enter output image file name: C:\Users\CHARANYA B\OneDrive\Desktop\SecureImageSteganography\images\secure_image.png
```

Figure 3.4(4): output image file path & name

- **Step 2: Password to AES Key Conversion**

Code:

```
def get_key(password):
return hashlib.sha256(password.encode()).digest()
```

Explanation:

- AES requires a fixed-length 256-bit key.
- SHA-256 generates a secure, irreversible key from any password.

```
PS C:\Users\CHARANYA B\OneDrive\Desktop\SecureImageSteganography> & C:\Python314\python.exe "c:/Users/CHARANYA B/OneDrive/Desktop/SecureImageSt
eganography/src/secure_encode.py"
Enter input image file name: C:\Users\CHARANYA B\OneDrive\Desktop\SecureImageSteganography\images\input_image.png
Enter output image file name: C:\Users\CHARANYA B\OneDrive\Desktop\SecureImageSteganography\images\secure_image.png
Enter password: secretkey123
```

Figure 3.4(5): password encoding

- **Step 3: Encrypt the Secret Message**

Code:

```
def encrypt_message(message, password):
key = get_key(password)
cipher = AES.new(key, AES.MODE_EAX)
ciphertext, tag = cipher.encrypt_and_digest(message.encode())
encrypted = base64.b64encode(cipher.nonce + tag + ciphertext).decode('utf-8')
return encrypted
```

Explanation:

- AES is used in EAX mode, which provides both:
  ✓ Confidentiality
  ✓ Integrity (authentication tag)
- The output contains:
  - Nonce (random value used once)
  - Tag (authentication)
  - Ciphertext
- Combined and Base64 encoded for safe embedding.

```
Enter secret message: this is an hidden message
[+] Data hidden successfully in C:\Users\CHARANYA B\OneDrive\Desktop\SecureImageSteganography\images\secure_image.png
PS C:\Users\CHARANYA B\OneDrive\Desktop\SecureImageSteganography>
```

Figure 3.4(6): Data hidden

Figure 3.4(7): Secure/Stego Image

➢ **Step 4: Convert Message to Binary**

Code:

```
def message_to_binary(message):
```

```
return ''.join(format(ord(i), '08b') for i in message)
```

Explanation:

- Each character is converted to 8 bits binary.
- Needed because LSB embedding works with bits.

➢ **Step 5: Embed Binary Data into Image Using LSB**

Code:

```
def hide_data(image_name, secret_data, output_image_name):
image = Image.open(image_name)
np_image = np.array(image)
total_bytes = np_image.size // 8
```

Explanation:

- Loads image into a NumPy array.
- Calculates maximum capacity (image pixels × 3 channels ÷ 8).

Embedding Process:

```
secret_data += '#####'
binary_secret_data = message_to_binary(secret_data)
data_index = 0
```

- A delimiter (#####) marks the end of hidden data.

Pixel LSB Replacement:

```
for values in np_image:
    for pixel in values:
        for n in range(3):
```

```
if data_index < len(binary_secret_data):
    binary_pixel = format(pixel[n], '08b')
    pixel[n] = int(binary_pixel[:-1] + binary_secret_data[data_index], 2)
    data_index += 1
```

Explanation:

- For every pixel and each R, G, B channel:
  - Take pixel value → convert to 8-bit binary
  - Replace last bit with message bit
  - Convert back → store into pixel

Save the Stego-Image:

```
encoded_image = Image.fromarray(np_image)
encoded_image.save(output_image_name)
```

- **Step 6: Running the Encoder**

  Code:

```
if __name__ == "__main__":
input_image = "input.png"
output_image = "encoded_image.png"
message = input("Enter secret message: ")
password = input("Enter password: ")
encrypted = encrypt_message(message, password)
hide_data(input_image, encrypted, output_image)
```

Explanation:

- User inputs are taken.
- Message → encrypted → embedded → saved.

B. IMPLEMENTATION OF DECODING (RECEIVER SIDE)

File: secure_decode.py

- **Step 1: Import Libraries**

Code:

```
from PIL import Image
import numpy as np
from Crypto.Cipher import AES
import hashlib
import base64
```

Explanation:

Same purpose as encode side.

- **Step 2: Password to AES Key Conversion (Same as Encode)**

Code:

```
def get_key(password):
return hashlib.sha256(password.encode()).digest()
```

- **Step 3: Decrypt Message**

Code:

```
def decrypt_message(encrypted, password):
key = get_key(password)
data = base64.b64decode(encrypted)
nonce, tag, ciphertext = data[:16], data[16:32], data[32:]
cipher = AES.new(key, AES.MODE_EAX, nonce=nonce)
```

```
message = cipher.decrypt_and_verify(ciphertext, tag)
return message.decode('utf-8')
```

Explanation:

- Base64 data is split into:
  - Nonce (16 bytes)
  - Tag (16 bytes)
  - Ciphertext (remaining)
- AES decrypts & checks authentication.

➢ **Step 4: Extract Binary Data from Image**

Code:

```
def reveal_data(image_name):
image = Image.open(image_name)
np_image = np.array(image)
binary_data = ""
for values in np_image:
for pixel in values:
for n in range(3):
binary_data += format(pixel[n], '08b')[-1]
```

Explanation:

- Reads LSB of every pixel channel.
- Collects bits into a binary string.

Convert Bits to Text:

```
all_data = binary_to_text(binary_data)
```

secret_data = all_data.split("#####")[0]

Explanation:

- Reads characters until ##### is found.

➢ **Step 5: Running the Decoder**

Code:

```
if __name__ == "__main__":
image_name = "encoded_image.png"
password = input("Enter password: ")
hidden_data = reveal_data(image_name)
decrypted = decrypt_message(hidden_data, password)
print("Hidden Message:", decrypted)
```

```
PS C:\Users\CHARANYA B\OneDrive\Desktop\SecureImageSteganography> & C:\Python314\python.exe "c:/Users/CHARANYA B/OneDrive/Desktop/SecureImageSteganography/src/secure_decode.py"
Enter encoded image name: C:\Users\CHARANYA B\OneDrive\Desktop\SecureImageSteganography\images\secure_image.png
```

Figure 3.4(8): Decoding Terminal

```
PS C:\Users\CHARANYA B\OneDrive\Desktop\SecureImageSteganography> & C:\Python314\python.exe "c:/Users/CHARANYA B/OneDrive/Desktop/SecureImageSteganography/src/secure_decode.py"
Enter encoded image name: C:\Users\CHARANYA B\OneDrive\Desktop\SecureImageSteganography\images\secure_image.png
Enter password: secretkey123
```

Figure 3.4(9): password decoding

```
PS C:\Users\CHARANYA B\OneDrive\Desktop\SecureImageSteganography> & C:\Python314\python.exe "c:/Users/CHARANYA B/OneDrive/Desktop/SecureImageSt
eganography/src/secure_decode.py"
Enter encoded image name: C:\Users\CHARANYA B\OneDrive\Desktop\SecureImageSteganography\images\secure_image.png
Enter password: secretkey123
[+] Hidden Message: this is an hidden message
```

Figure 3.4(10): Hidden Message is displayed

CHAPTER 4

EXPERIMENTATION & RESULTS

4.1 Description the data

The experimental evaluation carried out in this study relied on carefully selected data inputs to ensure a comprehensive assessment of the proposed system. Two main types of data were essential for this process: input images, which served as cover media for embedding hidden information, and input messages, which represented the content to be concealed during testing. These datasets were chosen to encompass a wide range of scenarios, allowing the system's performance, capacity, accuracy, and robustness to be thoroughly evaluated under varying conditions.In addition to these primary data categories, the study also incorporated the use of multiple passwords with different complexity levels to examine the behavior of the encryption component. By testing weak, moderate, and strong passwords, the evaluation was able to analyze how password strength influences AES key generation and overall security. A detailed breakdown of each data type and its purpose is provided in the following sections.

- **Input Images (Cover Images)**

  A diverse collection of digital images was utilized to thoroughly evaluate the algorithm's performance under different conditions. These images varied in resolution, file format, and content type to support capacity, accuracy, and robustness testing.

- **Input Messages**

  To assess the embedding algorithm across various scenarios, messages of different lengths and character types were used. These messages allowed analysis of capacity limits, accuracy of retrieval, and system performance. Table 4.1 summarizes the input message categories used.

Message Type	Length	Purpose
Short Text	**5–20 characters**	**Accuracy verification**
Medium-Length Text	**50–200 characters**	**Capacity evaluation**
Long Paragraph	**500+ characters**	**Performance and limit testing**
Special Characters	**Symbols, numbers, Unicode**	**Testing compatibility and encoding**

Table 4.1.1: Input Message Categories

- **Passwords Used for Encryption**

  For messages protected using AES encryption, various passwords were tested to observe differences in key generation and to ensure robustness across weak and strong passphrases. The embedding capacity remained unaffected by password complexity; however, stronger passwords resulted in more secure AES keys.

Password Example	Strength
12345	Weak
Pass@01	Moderate
Gf$29@xQ12!	Strong

Table 4.1.2: Password Strength Levels

4.2 Experimental Results

This section presents a comprehensive overview of the outcomes obtained from the experimental evaluation carried out to assess the effectiveness of the proposed steganographic method. The analysis focuses on three key aspects that determine the overall reliability and practicality of the system: the visual quality of the stego images, the accuracy with which hidden messages can be extracted, and the performance efficiency of the embedding and extraction processes. Through systematic testing with various image types, message lengths, and password complexities, the experiments provide valuable insights into how well the method preserves image integrity, maintains high extraction precision, and operates with minimal computational overhead. The following subsections summarize these findings in detail, demonstrating the robustness and efficiency of the proposed approach.

- **Visual Analysis**

A comparative assessment between the **original images** and the **stego images** revealed no perceivable visual differences. Even when embedding relatively large messages, the output images appeared visually identical to their originals.This is expected because the method modifies only the **Least Significant Bits (LSBs)** of pixel values, a process that introduces changes too minor for the human eye to detect. As a result, the visual integrity of the image is preserved.

- **Accuracy of Extraction**

The accuracy of message extraction was evaluated using messages of various lengths and character types. In all cases where the **correct password** was supplied, the extraction proceeded without error. When an incorrect password was used, decryption appropriately failed, demonstrating correct system behavior.

Test Case	Message Length	Result
Short message	**15 characters**	**Extracted 100%**
Medium message	**120 characters**	**Extracted 100%**
Long paragraph	**650 characters**	**Extracted 100%**
Special symbols	**—**	**Extracted correctly**
Wrong password	**—**	**Decryption failed (expected)**

Table 4.2.1: Extraction Accuracy Results

- **System Performance**

The performance of the steganography system was evaluated in terms of time efficiency, resource consumption, and image quality degradation. The results are summarized below. The low execution times and minimal memory consumption demonstrate the efficiency of the system. The **Peak Signal-to-Noise Ratio (PSNR)** greater than 50 dB further confirms that the embedding process introduces negligible distortion, ensuring high visual fidelity in the stego images.

4.3 Performance Analysis

- **Time Efficiency**

The proposed encoding algorithm demonstrates excellent time efficiency. When tested on standard 512×512 images, the complete process of encrypting the secret message using AES and embedding it into the cover image using LSB substitution finishes in less than one second on a typical modern computer.This rapid performance is due to the lightweight nature of LSB operations, which involve only simple bit manipulations. Additionally, AES encryption is highly optimized and can process even relatively large text inputs quickly.

- **Space Efficiency**

  The space overhead introduced by the steganography method is minimal. Since LSB techniques modify only the least significant bits of pixel values, the overall structure and size of the image remain nearly unchanged.In practice, the size of the resulting stego-image increases by only about 2–5 KB, which is negligible relative to the size of the original image. This minor increase is primarily due to metadata adjustments rather than the embedded message itself. Therefore, the approach maintains high space efficiency while still enabling secure data hiding.

- **Security Performance**

  The system provides strong security through a combination of cryptography and steganography. AES encryption ensures that even if an unauthorized party manages to extract or observe the embedded data, they cannot interpret or decrypt it without the correct password or key.On the other hand, LSB embedding helps conceal the very existence of the hidden message, making it difficult for attackers to detect that any secret data is present in the first place.

- **Robustness**

  In terms of robustness, the system performs reliably when using lossless image formats such as PNG. Since PNG preserves exact pixel values, the embedded LSB data remains intact and can be extracted without errors. Therefore, PNG is strongly recommended for embedding and extracting data, as it guarantees the preservation of the encoded information and ensures consistent performance.

Final Outputs (from Execution)

Encoding Output :

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS C:\Users\CHARANYA B\OneDrive\Desktop\SecureImageSteganography> & C:\Python314\python.exe "c:/Users/CHARANYA B/OneDrive/Desktop/SecureImageSteganography/src/secure_encode.py"
Enter input image file name: c:/Users/CHARANYA B/OneDrive/Desktop/SecureImageSteganography/images/input_image.png
Enter output image file name: c:/Users/CHARANYA B/OneDrive/Desktop/SecureImageSteganography/images/secure_image.png
Enter password: secretkey123
Enter secret message: Meet me at 6 PM behind the library
[+] Data hidden successfully in c:/Users/CHARANYA B/OneDrive/Desktop/SecureImageSteganography/images/secure_image.png
PS C:\Users\CHARANYA B\OneDrive\Desktop\SecureImageSteganography>
```

Figure 4.3.1: Encoded copy image is hidden

Decoding Output :

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS C:\Users\CHARANYA B\OneDrive\Desktop\SecureImageSteganography> & C:\Python314\python.exe "c:/Users/CHARANYA B/OneDrive/Desktop/SecureImageSteganography/src/secure_decode.py"
Enter encoded image name: c:/Users/CHARANYA B/OneDrive/Desktop/SecureImageSteganography/images/secure_image.png
Enter password: secretkey123
[+] Hidden Message: Meet me at 6 PM behind the library
PS C:\Users\CHARANYA B\OneDrive\Desktop\SecureImageSteganography> & C:\Python314\python.exe "c:/Users/CHARANYA B/OneDrive/Desktop/SecureImageSteganography/src/secure_decode.py"
Enter encoded image name: c:/Users/CHARANYA B/OneDrive/Desktop/SecureImageSteganography/images/secure_image.png
Enter password: secretkey
[!] Incorrect password or corrupted data
PS C:\Users\CHARANYA B\OneDrive\Desktop\SecureImageSteganography>
```

Figure 4.3.2: Decoded stego-image is not displayed due to incorrect password

```
PS C:\Users\CHARANYA B\OneDrive\Desktop\SecureImageSteganography> & C:\Python314\python.exe "c:/Users/CHARANYA B/OneDrive/Desktop/SecureImageSteganography/src/secure_decode.py"
Enter encoded image name: C:\Users\CHARANYA B\OneDrive\Desktop\SecureImageSteganography\images\secure_image.png
Enter password: secretkey123
[+] Hidden Message: this is an hidden message
```

Figure 4.3.3: Decoded stego-image is displayed, when password is entered correctly

CHAPTER 5

CONCLUSION & FUTURE SCOPE

Conclusion

The project titled "Secure Data Transfer Over Internet Using Image Steganography" successfully demonstrates a reliable, efficient, and practical method for ensuring secure communication in modern digital environments by integrating Advanced Encryption Standard (AES) with Least Significant Bit (LSB) Steganography, thereby achieving a powerful dual layer of protection—confidentiality through strong encryption and invisibility through seamless data hiding. In this system, the secret message is first encrypted using a password-derived key, ensuring that even if extraction is attempted by an unauthorized party, the content remains unreadable and protected. This encrypted data is then embedded into the LSBs of the cover image in a way that preserves the original visual quality, enabling the stego-image to blend naturally and be transmitted safely through any medium such as email, messaging apps, social networks, or cloud services without raising suspicion. Experimental results confirm the system's high accuracy in both embedding and extraction, and fast processing times under one second, all while maintaining cost-effectiveness by utilizing open-source tools and requiring no specialized hardware. Overall, the project fulfills its objectives by delivering a secure, scalable, user-friendly, and adaptable approach to protected data communication, making it suitable for diverse real-world applications including defense messaging, confidential corporate communication, secure personal exchanges, and safeguarding sensitive data across distributed systems. With further enhancements and adaptation to emerging technologies such as AI, blockchain, and IoT security, the system can evolve into an advanced, real-world communication platform suitable for highly sensitive environments.

Future Enhancement

Although the current system performs effectively for text-based secret messages embedded in images, there are several directions in which the project can be extended and enhanced. The future scope of this system includes the following:

- **Mobile and Desktop Application Development**

  The technology can be packaged into user-friendly Android/iOS mobile apps and Windows, Linux, or macOS desktop applications. A web-based platform can further improve accessibility and scalability. With intuitive interfaces, users will be able to easily hide, transmit, and extract secret data. This multi-platform availability supports both personal and enterprise-level use.

- **Integration with Cloud and IoT Platforms**

  The system can be integrated with cloud services to support secure remote data storage and exchange. IoT devices can use steganography to embed sensitive sensor data inside images to avoid interception. Smart city systems can communicate securely using hidden payloads in routine transmissions. This improves privacy, security, and reliability in automated environments.

- **AI-Based Adaptive Steganography**

  Machine learning can help identify optimal image regions for hiding data to minimize detectability. Deep learning models can avoid embedding patterns that might trigger steganalysis tools. AI can also adapt the embedding strategy based on image content for improved stealth. Additionally, intelligent systems can monitor and detect tampering or extraction attempts in real time.

- **Audio and Video Steganography**

  Data can also be hidden in audio signals and video frames. This massively increases storage capacity while improving stealth

REFERENCES

[1] ScienceDirect / Elsevier authors (2024) – A survey on Deep-Learning-based image steganography (Elsevier)

[2] DiffStega: Towards Universal Training-Free Coverless Image Steganography with Diffusion Models (2024)

[3] Robust Coverless Image Steganography Based on Human Pose Estimation (2024)

[4] Design of an Efficient Multimodal Image Steganography Framework with Multi-Domain Feature Analysis and Secret Sharing Operations (2024)

[5] JPEG Image Steganography With Automatic Embedding Cost Learning (2025)

[6] Digital Image Steganography and Reversible Data Hiding: Algorithms, Applications, and Recommendations (2025)

[7] Comparative Deep Learning Models in Applications of Steganography Detection (2025)

[8] Steganography Algorithm Based on Image Retrieval with SURF Features (2025) — Fan Li, Chenyang Liu, Zhenbo Dong, Zhibo Sun